POT: Codex Build Plan

ENGINEERING EXECUTION SPECIFICATION · VIBEATHON 2026

Build philosophy: Ship the complete MVP, communicate the larger vision.

We are not building the entire POT ecosystem for the hackathon. We are building one excellent, fully functional product experience that proves the concept:

Create a pot → Share it → Receive USDC contributions → Track progress → Settle the pot.

Everything outside this journey is either deferred or represented on the website as a clearly labelled future roadmap item.

The website must make POT look like a serious product with a long-term direction, but it must never pretend that planned features already exist.

1. Scope boundaries

Hackathon MVP

Build now

- Premium landing page.
- Wallet connection.
- Create a pot.
- Shareable public pot page.
- USDC contributions on Base.
- Contribution progress and history.
- Organizer dashboard.
- Payout and refund logic.
- Transaction verification.
- Clear product roadmap.

Post-hackathon

Do not build now

- Social activity feeds and reactions.
- Public pot discovery marketplace.
- Achievements and reputation.
- Creator and business accounts.
- Subscriptions and premium tiers.
- Referral rewards.
- Business API.
- Multi-chain support.

- Advanced analytics.
- Native mobile applications.

Important: The MVP must still be complete. Payouts, refunds, contribution accounting, contract security and error handling are not optional features.

2. Technical decisions

These are the default implementation choices. Codex should inspect the existing repository before changing its structure or replacing existing working infrastructure.

Layer	Choice
Frontend	Next.js App Router
Language	TypeScript
Styling	Tailwind CSS
UI	shadcn/ui, selectively
Wallet	wagmi + viem
Wallet onboarding	Coinbase Smart Wallet-compatible integration
Blockchain	Base
Development network	Base Sepolia

Settlement asset	Official Base Sepolia USDC
Smart contracts	Solidity
Contract framework	Foundry
Database	Supabase
Deployment	Vercel
Testing	Vitest + Foundry + Playwright

Do not invent contract addresses, token addresses, API keys or environment values. Verify official network configuration before use.

The blockchain is the financial source of truth. Supabase stores metadata and indexed activity, not authoritative balances.

Suggested repository structure

```
pot/
├── apps/
│   └── web/
│       ├── app/
│       │   ├── page.tsx
│       │   ├── dashboard/
│       │   ├── create/
│       │   ├── pot/[id]/
│       │   ├── roadmap/
│       │   └── layout.tsx
│       ├── components/
│       ├── hooks/
│       ├── lib/
│       └── styles/
├── contracts/
```

```
│   ├── src/
│   ├── test/
│   ├── script/
│   └── foundry.toml
├── supabase/
│   ├── migrations/
│   └── functions/
├── docs/
│   ├── PRODUCT.md
│   ├── ARCHITECTURE.md
│   ├── ROADMAP.md
│   ├── SECURITY.md
│   └── DEPLOYMENT.md
├── .env.example
└── README.md
```

This is a target structure, not permission to reorganize an existing repository unnecessarily.

3. Phased implementation plan for Codex

Each phase should be completed, tested and reviewed before moving to the next. Codex should not silently skip unfinished requirements.

Phase 0: Repository inspection and project foundation

Estimated: 0.5 day

Objective: Understand the existing project and establish a stable development foundation.

Tasks

- Inspect the repository, existing code, dependencies and deployment configuration.
- Identify existing components and reusable assets.
- Confirm whether the project already has a Next.js application.
- Establish the final architecture.
- Set up TypeScript, linting and formatting.
- Configure environment variable validation.
- Create the project documentation.
- Establish a consistent design token system.
- Configure Base Sepolia RPC and chain definitions.
- Configure Foundry if not already installed.

Required documentation

- `PRODUCT.md`: Product definition and MVP boundaries.

- `ARCHITECTURE.md`: Frontend, contracts, database and event flow.
- `ROADMAP.md`: Current and future capabilities.
- `SECURITY.md`: Threat model and security requirements.
- `DEPLOYMENT.md`: Development and production deployment.

Acceptance criteria

- Project builds successfully.
- Development server runs.
- No unnecessary dependency duplication.
- Environment variables are documented.
- Existing working functionality remains intact.

Phase 1: Brand, landing page and product experience

Estimated: 1 day

Objective: Build a premium website that communicates the product immediately.

This is both the marketing website and the first impression judges will have.

1.1 Brand implementation

Use the visual direction established in the product vision:

- Deep charcoal.
- Warm off-white.
- Electric lime accent.
- Premium typography.
- Minimalist layouts.
- Subtle motion.
- Strong financial data presentation.

The product should look like a serious consumer fintech application, not a generic Web3 template.

1.2 Landing page sections

Build the following:

Hero section

Headline:

Money moves better together.

Supporting copy:

Create a pot, bring your people together and reach shared goals with transparent USDC contributions on Base.

Primary CTA: `Create a POT`

Secondary CTA: `Explore how it works`

How it works

Three steps:

- Create your POT.
- Invite your people.
- Fund your goal together.

Use cases

- Group gifts.
- Trips and experiences.
- Community events.
- Shared purchases.

Product demonstration

Show an attractive product preview with realistic illustrative data. Clearly distinguish illustrative preview data from live financial activity.

Trust section

Explain:

- USDC contributions.
- Base settlement.
- Verifiable transactions.
- Transparent payout rules.

Roadmap section

Display the current MVP and future product stages. This is a real website section, not a placeholder.

Footer

- About.
- How it works.

- Roadmap.
- Documentation.
- Terms.
- Privacy.
- Contact.

Do not invent legal documents, audit certifications, partnerships or customer testimonials. Unavailable links should not pretend to work.

1.3 Website quality requirements

- Responsive on mobile, tablet and desktop.
- Accessible contrast and typography.
- Fast initial loading.
- SEO metadata.
- Social sharing metadata.
- Proper loading and error states.
- No broken buttons.
- No fake wallet balances.

Acceptance criteria

The landing page communicates POT within five seconds. A visitor understands what it does, why it matters and how to get started.

Phase 2: Wallet onboarding and application shell

Estimated: 0.5–1 day

Objective: Establish the user’s identity and wallet interaction.

Tasks

- Integrate wagmi and viem.
- Configure Base Sepolia.
- Support compatible wallets, including Coinbase Smart Wallet where supported.
- Add connect and disconnect functionality.
- Display connected wallet address.
- Detect unsupported networks.
- Provide network-switching functionality.
- Handle rejected connections.
- Handle wallet disconnections.
- Add clear wallet onboarding instructions.

Authentication

For the MVP, wallet-based identity is sufficient.

Do not introduce complicated email/password authentication unless required by the existing architecture.

The wallet address should be normalized and used as the user's blockchain identity.

User interface

Create:

- Navigation.
- Wallet connection button.
- Application layout.
- User dashboard shell.
- Transaction status notifications.
- Empty states.

Acceptance criteria

A user can connect a supported wallet, switch to Base Sepolia and navigate the application without errors.

Phase 3: Smart contract development

Estimated: 1–1.5 days

Objective: Build the financial engine before connecting the frontend to it.

Do not build simulated payment functionality and call it complete.

Contract architecture

Use a POT factory and individual POT instances, or a carefully justified alternative if a simpler architecture provides the same security and functionality.

Required functionality

Create POT

- Organizer address.
- Name and metadata reference.
- Target amount.
- Deadline.
- Refund/payout configuration.

- Contribution constraints.

Contribute

- Accept USDC.
- Validate pot state.
- Validate amount.
- Record contribution.
- Update total contributions.
- Emit contribution event.

Payout

- Enforce configured payout conditions.
- Enforce authorization.
- Prevent duplicate payout.
- Transfer funds safely.
- Emit settlement event.

Refund

- Validate refund eligibility.
- Prevent duplicate claims.
- Return the correct contribution amount.
- Emit refund event.

Critical design decision: avoid ambiguous settlement rules

For the MVP, implement two clearly defined pot types:

- Flexible pot: The organizer can withdraw according to the disclosed organizer-controlled rules.
- Target pot: The organizer can withdraw only after the target is reached; if the deadline expires below target, contributors can claim refunds.

If a pot has no deadline, its resolution rules must still be explicit.

Do not implement a vague combination of target, deadline and organizer discretion that could create conflicting states.

Security requirements

Codex must:

- Use OpenZeppelin where appropriate.
- Use safe ERC-20 transfer operations.

- Prevent reentrancy.
- Enforce state transitions.
- Prevent double payout.
- Prevent double refunds.
- Handle token decimals correctly.
- Reject invalid configurations.
- Validate all authorization paths.
- Test unusual state transitions.
- Test malicious and unexpected inputs.

Testing

At minimum, test:

- Successful pot creation.
- Invalid pot creation.
- Successful contribution.
- Zero contribution.
- Incorrect token.
- Contribution after closure.
- Payout before eligibility.
- Payout after eligibility.
- Unauthorized payout.
- Double payout.
- Refund before eligibility.
- Refund after eligibility.
- Double refund.
- Multiple contributors.
- Exact target reached.
- Target exceeded.
- Deadline expiration.
- Failed token transfer.

Acceptance criteria

The contracts pass their complete test suite and enforce every financial rule without depending on frontend validation.

No mainnet deployment until this phase has passed security review.

Phase 4: Database and blockchain event indexing

Estimated: 0.5 day

Objective: Make the application responsive without making the database the financial authority.

Supabase implementation

Create the required tables:

- `profiles`
- `pots`
- `contributions`
- `pot_members`
- `activities`

Keep the MVP database small.

Responsibilities

Supabase stores:

- Pot titles and descriptions.
- Cover image references.
- Organizer metadata.
- Public display settings.
- Indexed contract events.
- User preferences.

The blockchain determines:

- Actual balances.
- Confirmed contributions.
- Settlement status.
- Refund eligibility.
- Payout completion.

Indexing

Listen for contract events:

- PotCreated.
- ContributionMade.
- PotPaidOut.
- ContributionRefunded.

Use transaction hashes and log indexes as unique identifiers to prevent duplicate records.

Support indexing recovery and reconciliation. Do not rely exclusively on a browser session to record transactions.

Security

- Enable Row Level Security.
- Restrict organizer-only operations.
- Never expose the Supabase service role key.
- Validate user inputs server-side.
- Prevent unauthorized pot modification.
- Keep private user information separate from public pot metadata.

Acceptance criteria

The application can reconstruct pot activity from contract events and reconcile its displayed state against the chain.

Phase 5: Core POT application

Estimated: 1 day

Objective: Deliver the complete primary user journey.

This is the heart of the hackathon project.

5.1 Create POT page

Build a simple form containing:

- Pot name.
- Description.
- Target amount.
- Deadline.
- Pot type.
- Cover image, if supported.
- Terms and settlement explanation.

Show a clear summary before submission.

The application should validate the form before calling the contract.

5.2 Dashboard

Display:

- Created pots.
- Pots the user participates in.
- Total contributed.

- Total collected for the user’s active pots.
- Pending transactions.
- Recently updated pots.

Do not show invented or unverified financial totals.

5.3 Public POT page

This is the most important sharing destination.

Every pot must have a stable URL.

Example:

`pot.app/pot/{pot-id}`

The page should display:

- Pot name and description.
- Organizer.
- Funding goal.
- Confirmed amount collected.
- Progress bar.
- Deadline.
- Contribution count.
- Contribution history.
- Pot status.
- Contribution CTA.
- Settlement rules.
- Transaction explorer links.

The page must be accessible without connecting a wallet.

5.4 Contribution flow

- User opens a pot.
- User selects an amount.
- Application checks wallet and network.
- Application checks USDC balance.
- Application requests token approval when needed.
- User confirms contribution.
- Application displays pending state.
- Application waits for transaction confirmation.
- Confirmed contribution updates the UI.

Avoid unnecessary token approvals. Use an appropriate approval strategy and explain what the user is authorizing.

5.5 Settlement interface

Organizers should see the correct available actions based on the contract state.

Contributors should see their refund eligibility where applicable.

Do not display buttons for actions that the contract would reject.

Acceptance criteria

A second user can access a shared pot, contribute USDC and see the confirmed contribution reflected in the pot.

Phase 6: Roadmap, polish and demo readiness

Estimated: 0.5–1 day

Objective: Make the product presentation competition-ready.

6.1 Roadmap page

The roadmap must be visible from the landing page and accessible through the main navigation or footer.

Use four stages:

01. Core payments

Live MVP

Current build

- Create and share pots.
- USDC contributions on Base.
- Funding progress.
- Payout and refunds.
- Transaction verification.

02. Social experience

Planned

- Activity feeds.

- Reactions and milestones.
- Shareable achievement cards.
- Public pot discovery.
- User profiles and reputation.

03. Organizers and businesses

Planned

- Creator collections.
- Event and community collections.
- Organizer analytics.
- Premium management tools.
- Branded pot pages.

04. POT infrastructure

Future vision

- Developer API.
- Embedded collections.
- Merchant integrations.
- Programmable group payments.
- Additional network support.

These labels must reflect actual development status. Do not display future functionality as live, and do not invent release dates.

6.2 Final product polish

- Responsive layout checks.
- Loading skeletons.
- Empty states.
- Error boundaries.
- Toast notifications.
- Wallet rejection handling.
- Pending transaction handling.
- Failed transaction recovery.
- Transaction explorer links.
- Form validation.
- Accessible buttons and inputs.
- Mobile navigation.
- Consistent typography and spacing.

6.3 Demo preparation

Create a repeatable demonstration environment using testnet funds.

The demo should show:

- Organizer connects wallet.
- Organizer creates a pot.
- A second wallet opens the shared URL.
- Second wallet contributes USDC.
- Confirmed balance updates.
- Organizer executes the eligible settlement.
- Transaction is verifiable on Base Sepolia.

The demonstration must use real contract interactions, not mocked payments.

Phase 7: Deployment and submission

Final stage

Deployment checklist

- Deploy frontend.
- Configure Supabase production project.
- Configure verified network and token addresses.
- Deploy contracts to Base Sepolia for the demo.
- Verify deployed contracts.
- Configure environment variables.
- Test from a clean browser session.
- Test using two separate wallets.
- Confirm the public pot URL works.
- Confirm transaction links work.
- Check mobile responsiveness.
- Record demo.
- Finalize README and architecture documentation.

Mainnet decision

Do not rush to mainnet merely to make the submission look impressive.

Mainnet deployment is only appropriate if the contract, token configuration, financial rules, testing and security review are satisfactory. Testnet is an acceptable place to demonstrate a functional hackathon prototype.

4. Codex execution rules

The following instructions should govern every phase.

- Read the product vision and this build plan before implementation.
- Inspect existing code before making architectural changes.
- Work in small, verifiable increments.
- Do not generate the entire application in one uncontrolled implementation.
- Do not replace working features unnecessarily.
- Do not fabricate transaction data or claim mocked functionality is live.
- Never hardcode secrets.
- Never assume an external API, wallet or contract integration works without testing it.
- Do not introduce unnecessary dependencies.
- Run tests after each meaningful implementation.
- Fix errors before moving to the next phase.
- Update documentation when architecture or implementation changes.
- Maintain a clear list of completed and outstanding work.
- Do not implement deferred roadmap features during the MVP sprint.
- Prioritize financial correctness over visual shortcuts.

5. Phase completion tracker

Use this as the project execution checklist.

Build progress

0/8

Phase 0 · Repository and foundation

Inspection, architecture, docs, environment

Phase 1 · Brand and website

Landing page, product story, roadmap

Phase 2 · Wallet onboarding

Base Sepolia, wallet connection, app shell

Phase 3 · Smart contracts

Factory, contributions, payout, refunds, tests

Phase 4 · Database and indexing

Supabase, event indexing, reconciliation

Phase 5 · Core application

Create, dashboard, public pot, contribution, settlement

Phase 6 · Polish and demo

Roadmap, UX, mobile, end-to-end testing

Phase 7 · Deployment and submission

Deploy, verify, document, record demo

Copy checklist

6. Master instruction to give Codex

Use this as the initial instruction when starting the project.

POT — Autonomous Engineering Build Instructions

You are the lead software architect and senior full-stack engineer responsible for building POT.

POT is a consumer group-payment application built on Base. It enables users to create shared financial pots, invite participants, collect USDC contributions and settle funds through transparent smart-contract rules.

Your mission

Build a production-quality hackathon MVP that demonstrates the complete core user journey.

The MVP must support:

1. Wallet connection.
2. Creating a POT.
3. Generating a shareable public link.
4. Viewing a POT without connecting a wallet.
5. Contributing USDC on Base Sepolia.
6. Tracking confirmed contributions.
7. Displaying accurate funding progress.
8. Executing eligible payouts.
9. Claiming eligible refunds.
10. Verifying transactions on BaseScan.

The marketing website must also include a professional product roadmap showing the future direction of POT.

Critical scope restriction

Only implement functionality necessary for the hackathon MVP.

Do not implement social feeds, reactions, referral rewards, achievements, subscriptions, business APIs, multi-chain support or other deferred features.

However, create a polished roadmap section explaining these planned capabilities. Clearly distinguish live functionality from planned functionality.

Engineering requirements

- Inspect the existing repository before making changes.
- Preserve working code and existing project conventions.
- Use TypeScript, Next.js, wagmi, viem, Solidity, Foundry and Supabase where appropriate.
- Use Base Sepolia for initial development and demonstration.
- Verify official USDC token addresses and network configurations.
- Use the blockchain as the source of truth for financial state.
- Use Supabase for metadata and indexed activity.
- Never store private keys.
- Never fabricate financial activity.
- Never present mocked functionality as a live blockchain feature.
- Implement comprehensive smart-contract tests.
- Implement proper error handling and transaction states.
- Keep the application mobile-first.
- Avoid unnecessary dependencies and overengineering.

Execution protocol

Follow the eight phases in the project build plan:

Phase 0: Repository inspection and foundation.
Phase 1: Brand, landing page and product experience.
Phase 2: Wallet onboarding and application shell.
Phase 3: Smart contract development.
Phase 4: Database and blockchain event indexing.
Phase 5: Core POT application.
Phase 6: Roadmap, polish and demo readiness.
Phase 7: Deployment and submission.

Complete one phase at a time.

At the beginning of each phase:

- Inspect relevant existing code.
- Identify dependencies.
- State the intended implementation.
- Identify risks and unresolved decisions.

At the end of each phase:

- Run relevant tests.
- Run type checks and linting.
- Fix errors.
- Update documentation.
- Report completed work.
- Report outstanding work.
- Confirm acceptance criteria.

Do not mark a phase complete while critical acceptance criteria remain unmet.

Do not silently skip requirements.

Product quality bar

The finished application must feel like a serious consumer fintech product, not a generic Web3 template.

Visual direction:

- Deep charcoal.
- Warm off-white.
- Electric lime accents.
- Premium typography.
- Restrained animation.
- Clear financial data visualization.
- Excellent mobile experience.

The product’s primary message is:

“Money moves better together.”

The complete demonstration must show two separate wallets interacting with real deployed testnet contracts, including a real USDC contribution and valid settlement flow.

Security rule

Financial correctness takes priority over shipping speed.

Never deploy contracts handling real user funds without appropriate testing and security review. Do not claim an unaudited contract is audited or production-safe.

First action

Start with Phase 0.

Inspect the repository and report its current state. Then propose the implementation sequence based on what already exists.

Do not begin by generating the entire application at once.

My recommendation: Give Codex the product vision, the phased build plan and the master instruction together. Let it inspect the repository first. Then execute phase by phase, with particular attention to the contract’s payout and refund logic.

The hackathon-winning version is not the one with the most features. It is the one where the entire core experience works, looks exceptional and can be demonstrated convincingly.